Anonymous (Lambda) Functions

Introduction

An **Anonymous Function** is a function without a name. In Python, anonymous functions are created using the `lambda` keyword.

Because they are created using the `lambda` keyword, they are also called **Lambda Functions**.

They are usually used for short, simple operations that can be written in a single line.

Advantages

- Short and concise code
- No need to define a function using `def`
- Commonly used with `filter()`, `map()`, and `reduce()`
- Useful for quick calculations

Anonymous (Lambda) Function Syntax

Syntax

lambda parameters: expression

Components

- **lambda** → Keyword used to create an anonymous function.
- **parameters** → One or more input values (similar to function arguments).
- **expression** → A single expression that is evaluated and automatically returned. No `return` keyword is needed.

General Forms

No Parameters

lambda: expression

One Parameter

lambda x: expression

Multiple Parameters

lambda x, y: expression

Feature	Normal Function	Lambda Function
Keyword	`def`	`lambda`
Name	Usually has a name	Can be anonymous
Multiple statements	Yes	No (single expression only)
Return statement	Uses `return`	Returns automatically
Best for	Complex logic	Short operations

General Syntax

Normal Function Syntax

```python
def function_name(parameters):
    return value
```

Lambda Function Syntax

```python
variable_name = lambda parameters: expression
```

1. Adding Two Numbers

Normal Function

```python
def add(a, b):
    return a + b
result = add(10, 20)
print(result)
```

Lambda Function

```python
add = lambda a, b: a + b
result = add(10, 20)
print(result)
```

2. Square of a Number

Normal Function

```python
def square(num):
    return num ** 2
print(square(5))
```

Lambda Function

```python
square = lambda num: num ** 2
print(square(5))
```

3. Check Even or Odd

Normal Function

```python
def is_even(num):
    return num % 2 == 0
print(is_even(8))
```

Lambda Function

```python
is_even = lambda num: num % 2 == 0
print(is_even(8))
```

4. Find Maximum of Two Numbers

Normal Function

```python
def maximum(a, b):
    return a if a > b else b
print(maximum(10, 30))
```

Lambda Function

```python
maximum = lambda a, b: a if a > b else b

print(maximum(10, 30))
```

Anonymous (Lambda) Function Examples

Example 1: No Parameters

greet = lambda: "Welcome to Codegnan"
print(greet())
# Output:
# Welcome to Codegnan

Example 2: Square of a Number

square = lambda x: x * x
print(square(4))
# Output:
# 16

Example 3: Addition

add = lambda a, b: a + b
print(add(10, 20))
# Output:
# 30

Example 4: Maximum Number

```
largest = lambda a, b: a if a > b else b
print(largest(20, 10))
# Output:
# 20
```

Example 5: Even or Odd

```
check = lambda n: "Even" if n % 2 == 0 else "Odd"
print(check(8))
# Output:
# Even
```

Example 6: Product of Two Numbers

```
multiply = lambda a, b: a * b
print(multiply(5, 4))
# Output:
# 20
```

Example 7: String Length

```
length = lambda text: len(text)
print(length("Codegnan"))
# Output:
# 8
```

Filter Function

Definition

The `filter()` function is used to select elements from an iterable based on a condition. Only elements that satisfy the condition are returned.

Syntax

```
filter(function, iterable)
```

Returns

A **filter object** containing matching elements. Usually converted into a list.

Example 1: Even Numbers

```
numbers = [1, 2, 3, 4, 5, 6]
result = list(filter(lambda x: x % 2 == 0, numbers))
print(result)
# Output:
# [2, 4, 6]
```

Example 2: Odd Numbers

```
numbers = [1, 2, 3, 4, 5, 6]
result = list(filter(lambda x: x % 2 != 0, numbers))
print(result)
# Output:
# [1, 3, 5]
```

Example 3: Products Above ₹1000

```
prices = [500, 1200, 800, 2500, 600]
result = list(filter(lambda price: price > 1000, prices))
print(result)
# Output:
# [1200, 2500]
```

Example 4: Long Usernames

```
users = ["teja", "codegnan", "admin123", "raj"]
result = list(filter(lambda user: len(user) > 5, users))
print(result)
# Output:
# ['codegnan', 'admin123']
```

Map Function

Definition

The `map()` function applies a transformation to every element in an iterable.

Syntax

```
map(function, iterable)
```

Returns

A **map object** containing transformed values. Usually converted into a list.

Example 1: Square Numbers

```
numbers = [1, 2, 3, 4, 5]
result = list(map(lambda x: x * x, numbers))
print(result)
# Output:
# [1, 4, 9, 16, 25]
```

Example 2: Convert Names to Uppercase

```
names = ["teja", "ravi", "sneha"]
result = list(map(lambda name: name.upper(), names))
print(result)
# Output:
# ['TEJA', 'RAVI', 'SNEHA']
```

Example 3: Add GST

```
prices = [1000, 2000, 3000]
result = list(map(lambda x: x + (x * 0.18), prices))
print(result)
# Output:
# [1180.0, 2360.0, 3540.0]
```

Example 4: Calculate String Length

```
words = ["python", "java", "sql"]
result = list(map(lambda word: len(word), words))
print(result)
# Output:
# [6, 4, 3]
```

Reduce Function

Definition

The `reduce()` function reduces an iterable into a single value. It repeatedly applies a function to combine elements.

Syntax

```
from functools import reduce
reduce(function, iterable)
```

Important

`reduce()` is available in the `functools` module.

Example 1: Sum of Numbers

```
from functools import reduce
numbers = [1, 2, 3, 4, 5]
result = reduce(lambda a, b: a + b, numbers)
print(result)
# Output:
# 15
```

Working

1 + 2 = 3
3 + 3 = 6
6 + 4 = 10
10 + 5 = 15

Example 2: Product of Numbers

```
from functools import reduce
numbers = [1, 2, 3, 4]
result = reduce(lambda a, b: a * b, numbers)
print(result)
# Output:
# 24
```

Example 3: Largest Number

```
from functools import reduce
numbers = [10, 25, 8, 40, 15]
result = reduce(lambda a, b: a if a > b else b, numbers)
print(result)
# Output:
# 40
```

Example 4: Longest Word

```
from functools import reduce
words = ["python", "developer", "sql", "programming"]
```

```
result = reduce(lambda a, b: a if len(a) > len(b) else b, words)
print(result)
# Output:
# programming
```

Difference Between `filter()`, `map()`, and `reduce()`

Feature	`filter()`	`map()`	`reduce()`
Purpose	Select Elements	Transform Elements	Combine Elements
Output Size	Same or Smaller	Same Size	Single Value
Returns	Matching Elements	Modified Elements	One Final Result
Common Use	Filtering Data	Data Transformation	Aggregation